TMAP平台图生视频推理加速实践

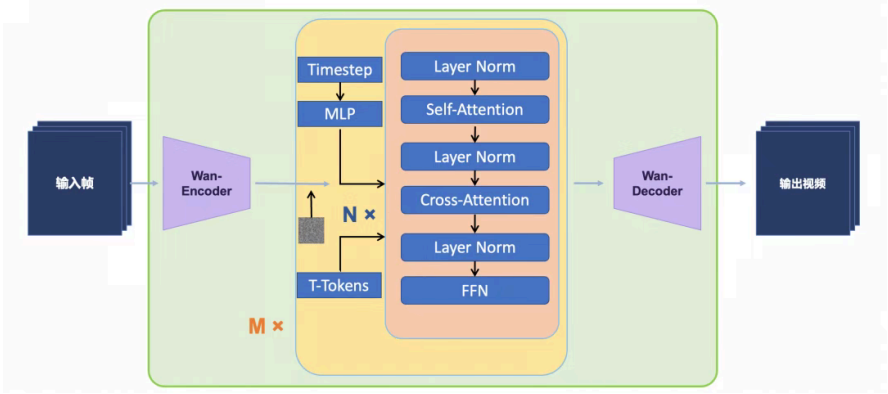
原创 用户&内容技术 大淘宝技术 2026年7月22日 01:04 浙江 3人

TMAP 平台针对图生视频模型在工业落地中面临的显存占用高、推理延迟大及成本昂贵等瓶颈，通过与“淘宝法象”团队合作，在多卡并行、Attention 加速、量化加速及缓存加速等领域进行了深入实践与优化。具体而言，平台采用 Deepspeed Ulysses 序列并行并结合通信重叠优化以降低单条请求延迟；引入 SageAttention 算子并设计层间选择性量化方案，在保持视频质量无损的前提下显著降低 Attention 耗时；通过线性层 W8A8/W8A16 混合量化解决显存溢出问题并消除 CPU Offload 开销；同时自研基于动态规划与路径感知成本张量的 DPCache 缓存方案，实现了比 TeaCache 和 TaylorSeer 更优的全局最优调度与加速效果。最终，该实践在生成质量肉眼不可区分的前提下，实现了除多卡外4.87倍的推理加速，将单次推理成本压缩至低于0.15元/秒。

01 背景

随着 AIGC 技术的爆发式发展，图生视频（Image-to-Video, I2V） 已成为当前多媒体内容生成领域最具潜力的方向之一。越来越多的开源与商业模型涌现，极大地降低了视频内容创作的门槛。

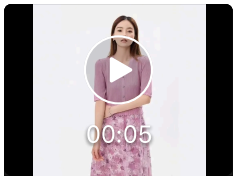
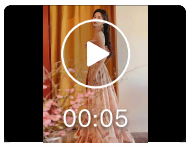
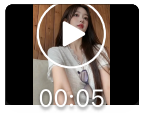
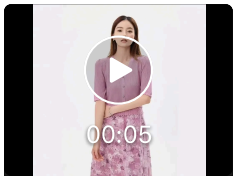
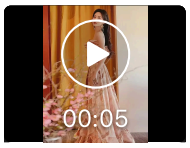
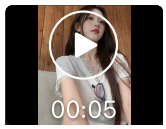
然而，图生视频模型在实际落地过程中面临着显存占用高、推理延迟高、硬件成本昂贵等瓶颈。在工业界，推理速度直接决定用户体验与服务成本。如何在保证生成质量的前提下，显著提升推理吞吐、降低单次生成延迟，是每一个将图生视频模型推向生产环境的团队必须攻克的核心课题。



TMAP 平台在与“淘宝法象”**视频生成算法团队**合作进行的图生视频推理加速落地实践中，在**多卡并行**、**Attention 加速**、**量化加速**、**缓存加速**等领域都进行了深入研究与落地实验，取得了在生成质量几乎无损（肉眼不可区分）的前提下，除多卡外 **4.87 倍**加速、推理成本压缩到低于**0.15元/秒**的显著成果。

02 加速效果展示

下面是目前图生视频主服务线上版本加速前后的生成效果对比，可以看出，除了偶尔出现的少量动作上的微小细节变化，基本能够做到肉眼不可区分的无损加速。

未加速			
加速后			

03 多卡通信优化

多卡介绍

从 1.5B 的 cogvideox 模型到 14B 的 wan2.1，从 360p 到 1080p 视频，从 5s 到分钟级视频，Scaling Law 广泛存在于视频生成领域中. 单卡的算力和显存逐渐不能满足时效性和可用性，多卡并行推理成为必需能力. 下表从能否提升吞吐和降低延迟的角度对比了主流的多卡并行策略:

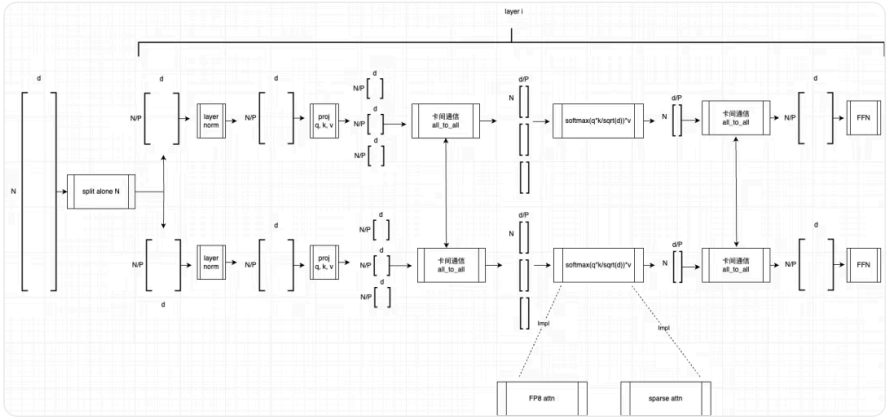
并行策略	核心机制	限制	能否提升吞吐	能否降低单条请求的延迟(不考虑排队)
Data Parallelism (数据并行)	每个设备持有完整模型副本，有多条独立数据, 每条在唯一设备处理.	一个设备必须能独立处理至少一条数据	可以. 平台容器化部署服务已经默认将不同请求并行了	不能.
Pipeline Parallelism (流水线并行)	按层将模型切分，不同设备处理不同层，数据像流水线一样流动.	需要尽可能消除“气泡”, 实现复杂	可以.	不能
Tensor Parallelism (张量并行)	层内权重切分到不同设备, 每条数据的激活过程(如矩阵乘法)由所有设备共同完成.	实现较复杂.	可以	可以
Sequence Parallelism (序列并行)	沿序列(token)维度切分输入, 相比DP而言切分发生在数据内部, 因此需要通信.	对ulysses, 序列长度需可被设备数整除; 对ring attn, attn算子和ring实现绑定.	可以	可以

多卡并行选型

我们服务的视频生成业务特点:

- 流量多为在线流量, 对堆积的容忍度较低;
- 同等质量下, 生成越快给用户的体感越好, 就越具有竞争力.
- 会结合多种加速策略

因此需要选择可以降低单条视频产出耗时的、易于和其他加速技术结合(量化, 缓存等)的多卡并行策略, 最终使用了 Deepspeed Ulysses 序列并行, 将线性层和注意力层的计算量均分到所有设备上. 下图展示了序列切分的过程:



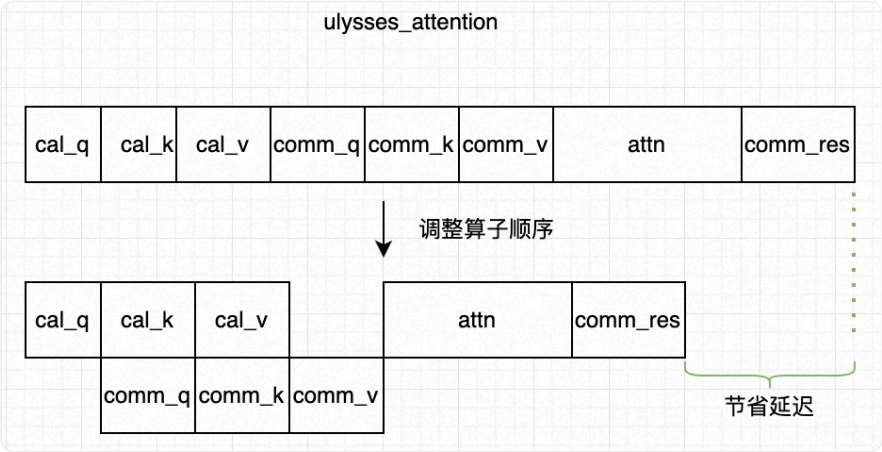
优化多卡并行

结合注意力量化算子 sageattention

选型时考虑了与注意力量化的结合, 由于 Ulysses 序列并行中注意力模块并行为 head 级并行, 全部 head 均分到各个设备上, 每张卡在序列维度完整, 这使得可以方便地接入高效注意力算子.

调整算子执行顺序掩盖通信

计算出本地的 q, k, v 后需要通过 all2all 通信传输后计算 attention, 在优化后期, all2all 的整体耗时已经接近 attn 2/3, 在分析 torch 的通信 api 后, 我们重构通信流程(如下图), 与前序计算尽量重叠.



效果

排除多流并行的开销后降低了25%的通信耗时(如下图).



04 Attention 性能优化

背景

单卡下各算子计算量(TFLOPs)的 profile 如下, 其中 attention 算子 aten._scaled_dot_product_flash_attention 占比73%; 如图左, 实际运行时, 默认使用的 flashattention2 算子耗时占比72%. 因此 attention 算子是尝试加速的首要方向.

```
Module                                                  FLOP     % Total
--------------------------------------------------  ----------  ---------
Global                                             98142.200T     100.00%
- aten.mm                                              4.742T       0.00%
- aten.bmm                                             0.103T       0.00%
- aten.convolution                                  1156.279T       1.18%
- aten._scaled_dot_product_efficient_attention        11.607T       0.01%
```

```
- aten.addmm                                    24575.255T      25.04%
- aten._scaled_dot_product_flash_attention      72394.214T      73.76%
```



attention--16bit



attention--8bit

attention量化前后耗时分布对比

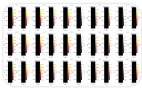
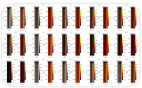
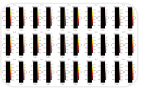
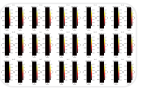
Flash-attention2算子在工程实现上已经没有显著提升空间, 进一步加速的思路就是降低计算量(稀疏)和降低数据宽度(量化). 在sageattention-1(24年底)提出之前, attention量化的开源方案只有flash-attention3. 但是FA3
不支持平台已有的计算卡架构, 而且sageattention论文提出FA3在视频生成模型上容易引入明显模糊.

attention量化

通过追踪学术界和开源社区动态, 确定了将结合可以达到加速与视频质量平衡的 sageattention 算子作为加速方向.

保持视频质量

业务的要求是加速前后的视频在全局和关键细节都要保持一致, 虽然 sageattention 已经很好, 但既然是量化就不可能完全无损. 如下视频, 在业务早期的 cogvideox 模型上直接使用会有闪现马赛克的 badcase. 如下图表所示, 我们通过可视化 attention 误差, 发现部分 attention 层(如 layer_2)在所有降噪迭代中误差都明显偏大, 而大多数层误差很小(如 layer_1, layer_3, layer_4). 最终设计了层间选择性量化(每次迭代时部分 attn 层保持原精度)的方案, 平衡了生成视频速度和质量. 在 Wan 模型视频质量受量化的影响更小, self-attention 模块的 attention 量化全局开启.

badcase	layer_1 over all iters	layer_2 over all iters	layer_3 over all iters	layer_4 over all iters
				

优化算子细节

per-channel 量化算子中的量化系数计算逻辑中(https://github.com/thu-ml/SageAttention/blob/d1a57a546c3d395b1ffcbeecc66d81db76f3b4b5/csrc/fused/fused.cu#L399), 如果某个 channel 数值全0会出现除0情况, 后续矩阵会出现 nan. 修复该逻辑后解决了业务 refiner 模型中生成黑屏视频的问题.

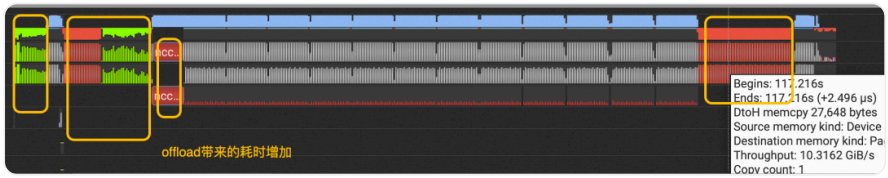
效果

量化后 attention 耗时降低到原1/3.

05 FFN性能优化

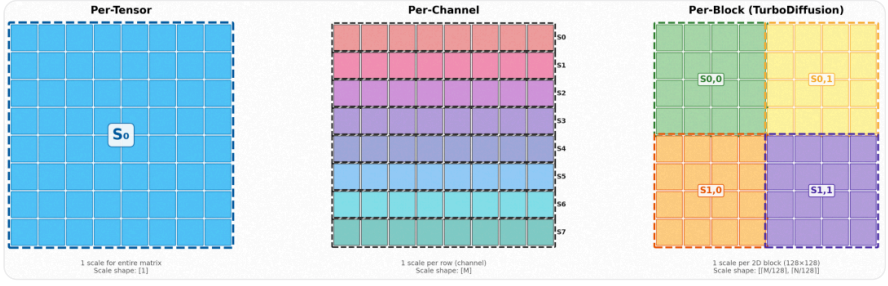
背景

在业务逐渐放量的背景下, 综合考虑性能、成本和供应后, 决定在上线时使用显存更低的计算卡, 这使得原先版本的模型无法在不 OOM 的情况下运行.早期开启了 cpu offload, 即每条视频生成都需要依次执行 text_encoder->image_encoder->transformer->vae 4组模型权重的 cpu 内存加载到 gpu、 gpu 推理、gpu 显存清空, 额外耗时显著增加(35s左右), 如下图. 此外从上节attention量化前后对比图也看到在 attn 算子通过 sageattention 量化加速之后, 线性层的耗时占比明显上升.



线性层量化

DiT的权重集中在线性层, 如果量化线性层权重从BF16到8比特可以降低10GB以上的显存占用; 同时如果计算也使用8比特, 即w8a8, rt和峰值显存也会降低. 常用的权重和输入的量化粒度包括per-tensor, per-channel, per-block等, 如下图所示.



我们采用了开源项目TurboDiffusion的 128*128 per-block量化算子, 相比torch _scaled_mm的per-tensor, per-channel量化算子, 在rt和视频生成细节的保持上都有更好效果.

然而线性层量化后同样增加了badcase, 我们同样采取了选择性量化的方案, 关闭部分量化. 在关闭对象上, 考虑到要保证非offload可以运行, 优先关闭激活而非权重量化. 最终我们保持权重的8比特量化, 在激活计算上降级部分w8a8到w8a16, 牺牲一些速度但是保持了视频质量.

效果

量化后无需运行时offload到cpu, 单条视频生成耗时从114s降低到84s.



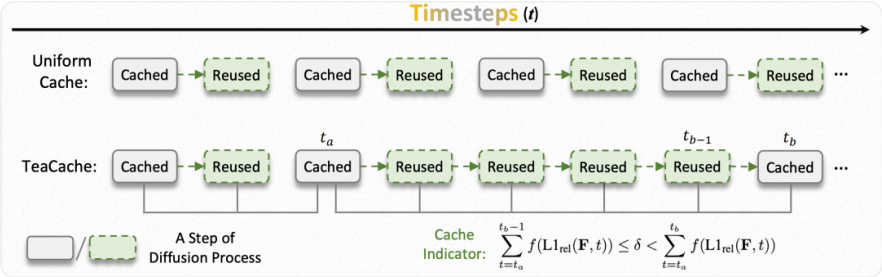
在图生视频（Image-to-Video）模型的推理过程中，DiT（Diffusion Transformer）模块占据了绝大部分的计算耗时。为了提升推理效率，**缓存加速**成为一种关键的技术路径。其核心思路是：针对占据推理时长比例最高的 DiT 模块，在特定时间步计算并存储Transformer的中间结果或输出，并在后续时间步中按策略复用，从而跳过或简化后续计算。

在缓存加速领域，我们深入探索了多种开源缓存方案，并最终基于业务需求自研了 **DPCache** 方案。该章节将简要分析开源方案 TeaCache、TaylorSeer 的原理、实现细节及实际效果，以及自研的 **DPCache** 如何解决现有痛点，实现质量与速度的双重突破。

TeaCache：基于输入差异的启发式缓存

核心原理

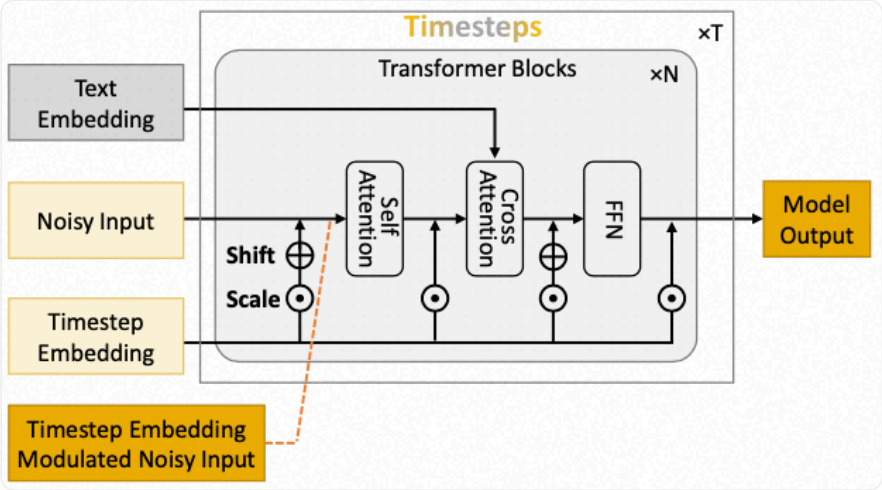
TeaCache 的核心思想是利用**模型输入的变化来预测模型输出的变化**。 理想情况下，若当前时间步 t 的输出 O_t 与之前某一步 t_a 的输出 O_{t_a} 差异很小，则可直接复用。但在推理过程中，只有完成了当前步的计算，才能进行这个比较，这就导致了“不计算就无法知道差异”的矛盾。为了解决这个矛盾，TeaCache 提出：模型输入与输出存在强相关性，且可以通过一个多项式对其进行拟合。



teacache论文原图（https://arxiv.org/pdf/2411.19108）

输入指标选择

TeaCache 选用**“时间步嵌入调制后的带噪输入”(Timestep Embedding Modulated Noisy Input)**。该指标既包含随时间变化的噪声信息，也融入了当前时间步 t 的信息，与模型输出相关性最强。



teacache论文原图（https://arxiv.org/pdf/2411.19108）

差异估计与重缩放

TeaCache 通过计算相邻时间步输入指标的相对 L1 距离来估计输出差异。为消除尺度偏差（Scaling Bias），引入了多项式拟合进行重缩放（Rescaling），将输入差异映射为更准确的输出差异估计值。这个校准过程是在少量样本上离线完成的，校准完成之后，能够得到适配校准集数据的多项式系数，可以在推理过程中直接使用，将当前时间步的输入差异映射为输出差异。

$$L1_{\mathrm{rel}}(\mathbf{O},t) = \frac{\|\mathbf{O}_t - \mathbf{O}_{t+1}\|_1}{\|\mathbf{O}_{t+1}\|_1}$$

推理流程

1. 在时间步 t_a 计算并缓存输出。

2. 对于后续时间步 t ，计算修正后的输入差异 $f(L1_{rel})$ 并累加。
3. 若累加和 $Sum < \delta$ （设定的阈值），则复用缓存；若 $Sum \geq \delta$ ，则进行完整计算并更新缓存。

$$\sum_{t=t_a}^{t_b-1} f(L1_{\text{rel}}(\mathbf{F},t)) \leq \delta < \sum_{t=t_a}^{t_b} f(L1_{\text{rel}}(\mathbf{F},t))$$

落地效果

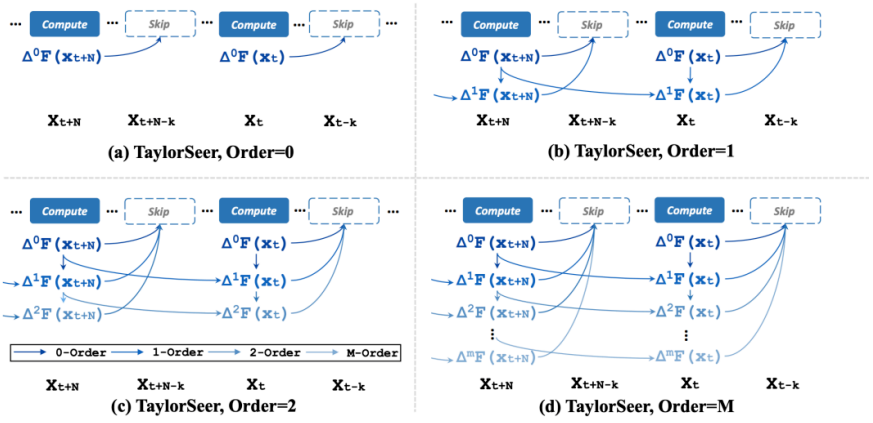
针对官方开源代码不支持 Diffusers 版本 CogVideo / Wan 2.1 模型的问题，我们自行完成了多卡适配及校准逻辑实现。

- **生成质量**：经人工标注对比，与 baseline 基本一致。
- **加速性能**：由于 TeaCache 在高加速比的情况下退化率较高，因此为了保证质量采用保守配置，在 8 卡环境下，**加速效果约为 50%-60%**。

TaylorSeer：从“缓存 - 复用”到“缓存 - 预测”

核心原理

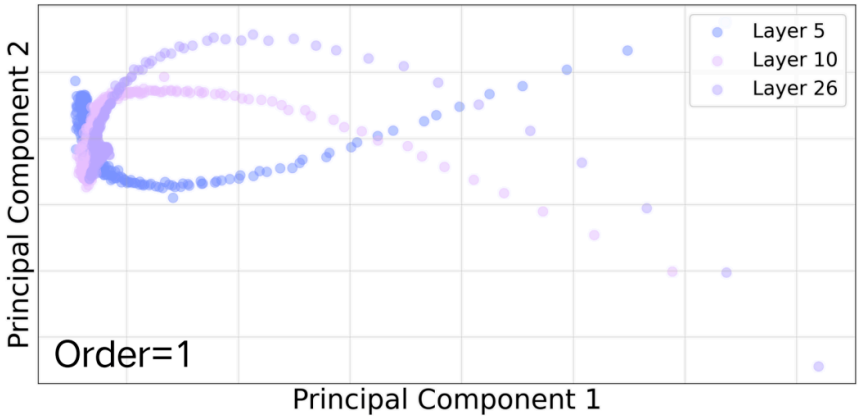
传统缓存方法在跳步较大时，仍然会直接复用多步前的缓存结果，这会导致误差累积严重。TaylorSeer 为了解决这个问题，将传统的缓存复用范式改良为 **“缓存 - 预测”**。



taylorseer论文原图（https://arxiv.org/pdf/2503.06923）

特征轨迹稳定性

TaylorSeer 通过 PCA 可视化发现，模型特征在不同时间步的演变形成了一条平滑、稳定的轨迹，意味着未来时间步的特征是可预测的。



taylorseer论文原图（https://arxiv.org/pdf/2503.06923）

泰勒展开预测

利用泰勒展开公式，通过部分历史点（按照固定间隔预先指定的完整计算时间步）的各阶导数（有限差分近似）来预测后续被跳过时间步的特征值。阶数越高，对曲线拟合越准确，理论预测准确度越高。

$$P_n(x_0;x) = P_n(x) = f(x_0) + \frac{f'(x_0)}{1!}(x-x_0) + \frac{f''(x_0)}{2!}(x-x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x-x_0)^n$$

落地优化

- **显存挑战**：TaylorSeer 原生实现需缓存每个 Transformer Block 中 Self-Attention、Cross-Attention 与 FFN 的结果及各阶差分。在 Wan 2.1 模型上，仅 1 阶实现就比 TeaCache 额外占用约 25G 显存，而 2 阶及更高阶数会占用更多的显存，导致模型完全无法在 4090 等消费级显卡运行。
- **优化效果**：经过实验，我们发现大部分缓存的中间特征是冗余的，对最终生成效果的影响很小。通过优化缓存特征位置，最终上线版本的显存占用比 TeaCache **低 3G 左右**，且生成质量相比 TaylorSeer 原版没有明显下降。

落地效果

- **生成质量**：由于加速比较高，在多数情况下虽然效果与 TeaCache 相近，但稳定性略差，badcase 比例稍高。
- **加速性能**：在 8 卡环境下，**加速效果约为 120%-140%**。
- **局限性**：只能按照预先设定好的固定间隔（如每 N 步）进行完整计算，这在大多数情况下都远非最优的采样序列。

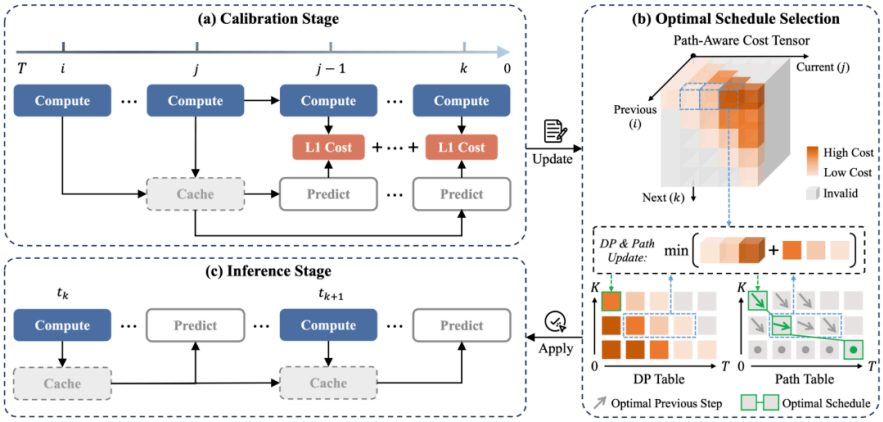
▌DPCache：基于动态规划与路径感知成本张量（Path-Aware Cost Tensor, PACT）的最优调度

针对 TaylorSeer 固定间隔不灵活、无法反映真实数据特征的问题，我们提出了 DPCache。该方案继承了“预测”范式，但通过**离线校准**与**动态规划**实现了全局最优的步数选择。

目前 DPCache 已入选 CVPR 2026 且完成代码开源，本文将简要介绍 DPCache的核心思路。

论文链接：https://arxiv.org/abs/2602.22654

代码开源地址：https://github.com/argsss/DPCache



核心思路

DPCache 的核心思路是通过模拟实际推理时的预测误差，利用动态规划找到全局误差最小的采样序列，并在推理时使用该序列与对应的预测器进行缓存与预测。基于该思路，DPCache 的整体流程可以分为三个阶段：离线校准、最优调度选择以及在线推理。

离线校准阶段：构建“路径感知成本”（PACT）

校准阶段的目标是计算如下成本数据：选择在某些时间步完整计算，并在中间步使用基于缓存的预测结果，造成的全局偏差。

具体步骤如下：

1. 对校准集执行完整采样，得到参考轨迹 从测试数据集中抽取少量能反映模型典型去噪轨迹特性的代表性输入，对每个校准样本，按原始 T 步采样流程完整运行，并在每个时间步记录模型对应推理时缓存位置的关键特征。
2. 评估“跳跃”偏差，构建 PACT 与仅衡量“从某一步跳到某一步”的简单误差不同，我们认为缓存预测的误差具有路径依赖性：预测中间时间步是否准确，不仅取决于跳跃的起始步，还取决于历史完整计算步（因为预测器通常要依赖最近的缓存状态或历史差分信息）。PACT 公式如下：

$$\mathcal{C}[i,j,k] = \sum_{\tau=k}^{j-1} \| \boldsymbol{h}_{\tau}^{L} - \boldsymbol{h}_{pred,\tau}^{L}(i,j) \|_1$$

最优调度选择阶段：用动态规划完成关键时间步序列计算

该阶段于线上初始化时进行，在给定只能完整计算 K 次的情况下，以 PACT 为成本，动态规划为最优路径算法，计算最优采样序列。该阶段的优化目标如下：

$$\arg\min_{\mathcal{T}} \sum_{m=2}^{K} \mathcal{C}[t_{m-1}, t_m, t_{m+1}]$$

动态规划递推公式如下：

$$D[m,k] = \min_{j>k} D[m-1,j] + \mathcal{C}\left[P\left[m-1,j\right], j, k\right]$$

其中 D[m, k] 为用 m 个关键步到达时间步 k 的最小累积成本，P[m, k] 则记录了达成该最小成本时的上一个关键步（用于回溯最优路径）。

该阶段得到的关键时间步序列与采样配置绑定，一旦生成，可在生产环境长期复用，直到配置变化才需要重新计算。

推理阶段：按关键时间步序列执行推理

在线推理时，对于每一个时间步执行如下策略：

- 关键步：执行完整模型前向计算，得到该时间步输出，并把指定特征写入缓存；
- 非关键步：调用与校准时完全相同的预测器，根据缓存快速生成该时间步所需特征。

落地效果

- **生成质量**：生成结果退化比例明显低于 TeaCache 及 TaylorSeer。
- **加速性能**：由于 DPCache 能够通过校准找到最优的采样序列，因此可以进一步减少完整计算步数，**加速效果约为 140%-160%**。

已落地方案对比

对于我们在探索过程中已落地的三种缓存策略。下表从多个维度进行了综合对比：

	TeaCache	TaylorSeer	DPCache
核心思路	使用输入差异预测模型输出差异，差异累积超过阈值时进行完整计算，否则复用缓存	固定间隔全量计算，利用缓存历史通过泰勒展开预测中间步。	利用预测误差累积构建成本矩阵，基于 DP 求解最小累积误差路径。
需要离线校准	需要	不需要	需要
生成质量损失	中等	较低	低
加速比例	中等	较高	高
优点总结	基础实现简单	无需校准，通用性强，加速效果优秀	加速效果最优，生成质量高，步数选择灵活
缺点总结	需自行实现校准逻辑，加速效果与生成质量均一般	固定间隔跳步，灵活性差，显存占用高	校准逻辑实现复杂

通过对三种缓存方案的实践与对比，可以得出以下结论：

1. **TeaCache** 是视频生成缓存加速领域的经典方法，但在高质量生成场景下，其启发式阈值难以平衡质量与速度。
2. **TaylorSeer** 证明了“预测范式”的有效性，大幅提升了加速比，但固定间隔限制了其进一步优化空间，且额外的显存占用对实际落地影响较大。
3. **DPCache 在大多数情况下**是更好的选择。它通过**路径感知成本**与**动态规划**的结合，从而找到全局最优的计算路径。在线上AB测试中，DPCache 不仅继承且进一步提升了 TaylorSeer 的高加速比，且在灵活性及生成质量稳定性上实现了突破。



为了能够在显存规格较低的显卡上稳定运行高显存要求的视频生成模型，我们在很多方面进行了尝试，之前提到的很多加速实践方向都包含了针对显存的特殊优化，在这一章节将进行详细介绍与汇总。

量化

显存占用来源包括模型参数和运行时的中间结果. 视频生成模型的参数集中在线性层，前面提到的线性层量化将模型参数中transformer占用显存降低近50%；运行时显存主要来自self-attention和线性层，其中self-attention计算全部从16比特量化成8比特，线性层的激活有近一半量化到了 8 比特.

缓存

我们在落地开源缓存方法的过程中，多数主流缓存策略在推理时都需要占用大量的额外显存，以提出了基于预测进行缓存的taylorseer为例，对于wan2.1模型，仅使用一阶预测，推理时的额外显存占用就将近40g，比teacache多37g，这对于想在4090上进行部署的服务来说是完全不可以接受的。所以，在后续dpcache的开发与落地过程中，我们对缓存算法的显存占用做了两个方面的优化:

优化缓存位置

taylorseer对于Transformer block内部的每个layer都进行了缓存，因此导致了大量的额外显存占用，但经过实验对比，实际上只对特定block的输出进行缓存，即可在几乎没有额外损失的前提下，实现显存占用的显著降低。为了进一步权衡效果与显存占用，我们还支持了自适应的block级多点缓存，可以根据实际情况进行选择。在单block输出缓存的情况下，二阶预测的显存占用甚至低于teacache。

block级offload

有一定限制，特定情况下可以使用，借助fsdp自带的预加载能力，当使用block级dpcache缓存时，对于非全局完整计算的block使用offload，即可显著降低显存占用。offload带来的额外IO与通信开销在不同显卡上会有区别，但因为只对非完整计算的block进行，因此对整体推理延迟的影响相对较低，也可根据实际情况权衡offload的block比例。

08 总结与展望

以上只是过去一年 TMAP 平台在推理加速领域落地成果的一部分。除了 wan系列视频生成与编辑模型，也涉及 qwen-image、flux 等图片生成与编辑模型。这些合作项目的成功落地，离不开AI原生算法、 直播数字人等合作团队的倾力相助。

未来，我们将继续深耕推理加速领域，探索量化、稀疏化等方向的先进技术，也会进一步拓展 TMAP 的内容能力覆盖范围，为集团更多内容场景提供高效、可扩展的算力底座。

09 团队介绍

本文作者蓝愿、行宣、不竭，来自淘天集团–用户&内容技术团队。团队聚焦视频、直播、图文、音乐等集团内容资源，建设兼具技术先进性与成本竞争力的内容能力矩阵，统一内容库并贯穿内容生产、处理与分发全链路，服务淘宝各场域。

其中围绕直播与内容理解、音视频转码与增强、视频AIGC、直播数字人等AI方向，团队打造了统一内容算力平台 TMAP，提供 AI 时代下的大规模服务部署、算力调度优化、推理加速等能力，以先进、高效的技术底座支撑内容能力的规模化落地，持续提升内容生态的运行效率与服务体验。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 803

留言

写留言

还没有留言，来写首评